

PHASE 4A – AI Evaluation Engine (Technical Design, Code & Implementation)

1. Phase Objective

Phase 4A focuses on **building the complete rule-based AI evaluation engine** for UTAP. This phase establishes the **entire algorithmic backbone** using deterministic logic before introducing ML/NLP.

This document includes: - Exact commands used - Folder structure - Models - Algorithms - Code written in each file

No training, testing, or deployment is done in this phase.

2. Architectural Decision: Separate AI Engine

Why separate app?

- Avoid breaking Phases 1–3
- Safe experimentation
- Clean integration later

Command used

```
python manage.py startapp ai_engine
```

Added to settings

```
INSTALLED_APPS += ['ai_engine']
```

3. Folder Structure Created

```
ai_engine/  
├── models.py  
├── services/  
│   ├── text_utils.py  
│   ├── skill_extractor.py  
│   ├── registry.py  
│   ├── scoring.py  
│   └── ranking.py
```

4. Database Models (Exact Code)

4.1 Resume Model

File: `ai_engine/models.py`

```
from django.db import models
from django.conf import settings
from django.utils import timezone

class Resume(models.Model):

    candidate=models.ForeignKey(settings.AUTH_USER_MODEL,on_delete=models.CASCADE,related_name='ai')
    file=models.FileField(upload_to='resumes/')
    raw_text=models.TextField(blank=True)
    uploaded_at=models.DateTimeField(default=timezone.now)

    def __str__(self):
        return self.candidate.email
```

4.2 SkillRegistry Model

```
class SkillRegistry(models.Model):
    name=models.CharField(max_length=100,unique=True)
    frequency=models.IntegerField(default=1)
    confirmed=models.BooleanField(default=False)
    last_seen=models.DateTimeField(default=timezone.now)

    def __str__(self):
        return self.name
```

4.3 SkillCluster Model

```
class SkillCluster(models.Model):
    name=models.CharField(max_length=100)
    skills=models.ManyToManyField(SkillRegistry,related_name='clusters')

    def __str__(self):
        return self.name
```

Migration Commands

```
python manage.py makemigrations ai_engine
python manage.py migrate
```

5. Text Normalization Algorithm

File: `ai_engine/services/text_utils.py`

```
def normalize_text(text):
    text=text.lower()
    cleaned=[]
    for ch in text:
        if ch.isalnum() or ch==' ':
            cleaned.append(ch)
        else:
            cleaned.append(' ')
    normalized=''.join(cleaned)
    while '  ' in normalized:
        normalized=normalized.replace('  ',' ')
    return normalized.strip()
```

Purpose: - Remove noise - Ensure deterministic tokenization

6. Skill Extraction Algorithm

File: `ai_engine/services/skill_extractor.py`

```
CONTEXT_KEYWORDS=('skill','skills','experience','experienced','using','worked','with','in')
STOP_WORDS=('and','or','the','a','an')

def looks_like_technical_token(word):
    if len(word)<3:
        return False
    for ch in word:
        if ch.isdigit():
            return True
    if '-' in word or '+' in word:
        return True
    suffixes=('js','db','ml','ai','sql','api','apis','net','flow')
    for suf in suffixes:
        if word.endswith(suf):
            return True
    return False
```

```

def extract_skill_candidates(normalized_text):
    words=normalized_text.split(' ')
    skills=set()
    capture=False
    for word in words:
        if word in CONTEXT_KEYWORDS:
            capture=True
            continue
        if word in STOP_WORDS:
            capture=False
            continue
        if looks_like_technical_token(word):
            skills.add(word)
            capture=True
            continue
        if capture:
            skills.add(word)
        else:
            capture=False
    return skills

```

7. SkillRegistry Learning Logic

File: `ai_engine/services/registry.py`

```

from django.utils import timezone
from ai_engine.models import SkillRegistry

RECRUITER_BOOST=5
CONFIRM_THRESHOLD=10

def normalize_skill(skill):
    return skill.strip().lower()

def update_from_recruiter(skill_list):
    for skill in skill_list:
        name=normalize_skill(skill)
        if not name:
            continue
        obj,created=SkillRegistry.objects.get_or_create(name=name)
        if created:
            obj.frequency=RECRUITER_BOOST
            obj.confirmed=True
        else:
            obj.frequency+=RECRUITER_BOOST

```

```

        obj.confirmed=True
    obj.last_seen=timezone.now()
    obj.save()

def update_from_resume(skill_set):
    for skill in skill_set:
        name=normalize_skill(skill)
        if not name:
            continue
        obj,created=SkillRegistry.objects.get_or_create(name=name)
        if created:
            obj.frequency=1
            obj.confirmed=False
        else:
            obj.frequency+=1
            if not obj.confirmed and obj.frequency>=CONFIRM_THRESHOLD:
                obj.confirmed=True
        obj.last_seen=timezone.now()
        obj.save()

```

8. Scoring Algorithm

File: `ai_engine/services/scoring.py`

```

from ai_engine.models import SkillRegistry,SkillCluster

EXACT_MATCH_WEIGHT=1.0
CLUSTER_MATCH_WEIGHT=0.7
RELATED_MATCH_WEIGHT=0.4

def get_skill_clusters(skill_name):
    try:
        skill=SkillRegistry.objects.get(name=skill_name)
    except SkillRegistry.DoesNotExist:
        return set()
    clusters=SkillCluster.objects.filter(skills=skill)
    related_skills=set()
    for cluster in clusters:
        for s in cluster.skills.all():
            related_skills.add(s.name)
    return related_skills

def calculate_match_score(job_skills,resume_skills):
    if not job_skills:
        return 0
    total_score=0.0

```

```

max_score=len(job_skills)*EXACT_MATCH_WEIGHT
resume_set=set(resume_skills)
for job_skill in job_skills:
    job_skill=job_skill.lower()
    if job_skill in resume_set:
        total_score+=EXACT_MATCH_WEIGHT
        continue
    related_skills=get_skill_clusters(job_skill)
    if related_skills and resume_set.intersection(related_skills):
        total_score+=CLUSTER_MATCH_WEIGHT
        continue
    for res_skill in resume_set:
        if job_skill in res_skill or res_skill in job_skill:
            total_score+=RELATED_MATCH_WEIGHT
            break
percentage=(total_score/max_score)*100
return round(percentage)

```

9. Eligibility-First Ranking Logic

File: `ai_engine/services/ranking.py`

```

MIN_MATCH_SCORE=20

def evaluate_eligibility(application):
    if application.score<MIN_MATCH_SCORE:
        application.status='rejected'
    else:
        application.status='eligible'
    application.save()

def rank_eligible_applications(applications):
    eligible=[]
    for app in applications:
        if app.status=='eligible':
            eligible.append(app)
    eligible.sort(key=lambda x:x.score,reverse=True)
    return eligible

```

10. What Phase 4A Delivers

- Complete rule-based evaluation engine
- Dynamic skill learning
- Percentage-based scoring
- Eligibility-first rejection

- Non-competitive ranking
 - Fully explainable logic
-

11. What Is NOT Done in Phase 4A

- No ML training
- No dataset ingestion
- No NLP embeddings
- No deployment

These are deferred to Phase 4B and Phase 4C.

12. Phase 4A Final Outcome

Phase 4A establishes a **stable, deterministic, and extensible AI evaluation engine** that serves as the ground-truth system for future training, testing, and deployment phases.